Project Bridge — path to production

Andrés Molina · 2026-10-15 · Part 3 of the Clara Finance Engineer case · figures from the tool's run on the case data pack · detail in the annex

Project Bridge is one Python program that reads the data pack and writes a page plus nine files: 262 exceptions, 11 proposed entries, none posted (`run_manifest.json`). Proof the entries are right: after all of them the Mastercard payable (2105) is a credit of USD 45,572.30; the 30-Sep settlement debited on 1-Oct is USD 44,643.99 (BS0031); the USD 928.31 between them is exactly the month cut-off rows. The corrected payable ties to the next day's cash.

Data access and refresh

A read-only service account pulls each source: Minsait's daily file, Mastercard clearing and summary, the bridge account statement, the NetSuite journal extract. Where only a portal download exists (T01, T03, T04), a named person drops the file in one controlled folder and the run fingerprints it. The pack holds tokens and last 4 only, and a pre-check confirms no card numbers. **NetSuite requirement:** the journal extract must carry who created and who approved each entry; today it does not, so "preparer ≠ approver" cannot be checked on any posted entry.

Security with AI tools

The reconciliation uses no AI: it is tested code, so the same inputs give the same figures. The code was written with AI through reviewed commits, each step recorded in the AI usage log. The agents proposed below would read free text nobody at Clara wrote (email, bank narratives), which can carry instructions; that is contained by what they can do, not by trusting them: read-only tools, every figure cited to a file and row, any write queued for human approval, each question and answer logged with the run it read, as the policy requires of AI-assisted workpapers. The model key stays on a server behind company login with a spend cap; no customer data reaches a provider InfoSec has not approved.

Controls and audit trail

- **Propose, never post.** A person posts in NetSuite after approval. The ERP import file (`proposed_journal_lines.csv`) uses the ledger's own columns, with ids prefixed `PROPOSED-` and source `PROPOSED_NOT_APPROVED`, so it cannot pass for approved entries.
- **Approval limits:** USD 10,000+ Controller, USD 100,000+ CFO (4 of 11 entries need the Controller). The tool leaves preparer and approver empty; in production they fill from the login, and approval refuses the preparer.
- **Every figure traces to a source row** (`src` column, file and line).
- **Pre-checks before trust.** 13 run first; 5 fail on this data, including two processor days with a missing or misdated ledger entry and the missing processor invoice. The page opens on them as a Data quality table. Today a failure warns; in production a missing input blocks "ready for review".
- **Run manifest:** code commit, run time, SHA-256 of every input, headline totals. Production adds who ran and reviewed it, and keeps each run write-once.

Monitoring

The daily run alerts on failed pre-checks, items open over 5 business days (121 here) and any cash break. A missing file is reported, never estimated. Each action states the USD it resolves,

counted once: of USD 567,345.29, USD 167,345.29 needs action by Clara and USD 400,000.00 is funding in transit.

Two findings beyond the reconciliation

- **Customers were billed on processor errors.** September statements went out from a receivable built on the processor feed: refunds billed as purchases, duplicates billed twice, a stale BRL rate, five transactions never billed (USD 9,009.43 across the four). The entries fix the ledger, not the customer; Billing re-bills from the exception rows.
- **The USD 400,000 gap in bank cash is prefunding in transit**, proved by three sources (the Controller's entry, her email, the 1-Oct bank credit with the same reference), with three caveats: the outflow is unverified, it is booked in bank cash rather than cash-in-transit, and October can double-count it.

Owners, hand-off and adoption

Owners come from the close checklist (Finance Ops reconciles, Treasury owns cash, the Controller approves, the CFO locks the period); the email thread names the person for 16 of 26 tasks, and the role stays the owner of record. **Engineering takes** connectors, scheduling, accounts, the shared store and alerting. **Finance keeps** the rules and tests; a rule change needs the Controller. No Engineering ask until the shadow month proves value.

1. **Shadow month: the October close**, beside the spreadsheet; every difference explained.
2. **Controller first.** A missing processor entry, one posted a day late and a duplicate bank debit left the payable at a debit of USD 70,588.66; a daily run flags the entries the next day, not on working day 3. The invoice accrual comes itemised.
3. **Treasury.** The cash reconciliations it does by hand (T06, T07), plus days of coverage (26.11 at 30 Sep) and an illustrative weekly transfer suggestion.

Known gaps, with the fix

Analyst notes live in one browser (fix: a shared table keyed by business reference). Each run stands alone (fix: compare runs, with age). Email and Slack sit outside the tool (fix: replies read into the exception propose a status change the owner accepts; they never close or post by themselves). **Cut to fit the time:** logins and shared notes; written, not built: live connections, the margin view and the agents below. Posting is out by design, never by time.

Optional: path to a day-zero close (proposal, not built)

A day-zero close means September closes on 1 October because every day was already reconciled. Two agents:

1. **An agent that answers questions about the close** ("why is the payable a debit?"), each figure cited to its file and row, or it says it cannot find it. It writes nothing.
2. **A continuous-improvement loop.** After each close, an agent reads the analysts' corrections and their reasons and proposes new rules. Each rule comes **with its test**, built from the rows that motivated it, and the Controller approves it before it runs: the process never changes its own rules.